



41. Gaussian Mixture Models

Perform clustering by utilizing probability distribution?

Previous Section:

 [40. Hidden Markov Models](#)

Contents:

[1. Gaussian Mixture Models - The Overview](#)

[2. Gaussian Mixture Models - The Comprehensive Workflow](#)

[3. Gaussian Mixture Models - The Expectation–Maximization Algorithm](#)

[4. Gaussian Mixture Models with Python](#)

[Summary](#)

[References](#)

You’ve probably encountered the Gaussian distribution before—well it is just a fancy way of calling **normal distribution**, in which there is the classic bell curve that quietly appears everywhere in statistics and machine learning. In most beginner examples, we deal with a single clean curve, representing one underlying population with a single center of behavior.

But reality is rarely that clean. So the natural question arises: what happens if the data doesn’t follow just one bell curve—but several overlapping ones? Does that mean the model is “broken” or the distribution is invalid? Not quite.

From a more observational standpoint, what we’re really seeing is not a failure of structure—but the presence of multiple hidden structures coexisting in the same space. Instead of forcing data into one rigid cluster or one dominant pattern, we can assume something more flexible: that each data point may

belong to multiple probabilistic “sources,” each shaped like a Gaussian distribution.

This is where Gaussian Mixture Models begin to matter. Rather than asking, *“Which group does this point strictly belong to?”* we shift the question to:

“How likely is it that this point was generated by each underlying distribution?”

In other words, clustering stops being a hard assignment problem—and becomes a process of interpreting overlapping probabilities.

From my perspective, this is less about separating data into boxes, and more about reconstructing the unseen generative story behind it: multiple hidden distributions, blending together, each quietly contributing to what we observe as a single dataset.

We don’t force separation. We infer structure. And that is where the model begins to take shape.

1. Gaussian Mixture Models - The Overview

A Gaussian Mixture Model (GMM) is a probabilistic, soft-clustering method that assumes data is not generated from a single source, but from a mixture of multiple hidden Gaussian (normal) distributions. Each of these distributions has its own unknown parameters, and together they explain how the dataset is formed.

Instead of forcing every data point into a single, fixed cluster, GMM takes a different approach: it treats clusters as probability distributions. So rather than asking *“Which group does this point belong to?”*, it asks a softer, more realistic question: *“How likely is it that this point came from each group?”*

This is where GMM starts to feel fundamentally different from methods like K-Means.

K-Means performs **hard clustering**. It draws strict boundaries in space and assigns each point to exactly one cluster based on distance. There is no ambiguity—one point, one cluster, end of story.

GMM does not accept that simplicity. Instead, it models uncertainty directly. Each point can belong to multiple clusters at the same time, each with a different probability. These probabilities are learned using the **Expectation-**

Maximization (EM) algorithm, an iterative process where the model alternates between estimating cluster membership and updating the parameters of each Gaussian distribution until everything stabilizes.

So instead of drawing boundaries, GMM tries to reconstruct the *hidden probability landscape* that generated the data in the first place.

Key properties of Gaussian Mixture Models

- **Soft Clustering:** GMM does not force a decision. A single data point can belong partially to multiple clusters. For example: *“Point A has 70% probability of belonging to Cluster 1 and 30% to Cluster 2.”* → This makes the model more honest about uncertainty, especially when clusters overlap.
- **Flexible Shapes:** K-Means assumes clusters are spherical because it relies on simple distance to a center. GMM removes that restriction. Each Gaussian has its own covariance structure, which allows clusters to stretch, rotate, and deform into elliptical shapes. This makes it far more expressive in real-world data.
- **Bell Curve Assumption (but in plurality):** Instead of one distribution, GMM assumes the data is generated from several Gaussian distributions mixed together. Each one has a mean and covariance, and the final dataset is the weighted combination of all of them. The goal of the model is not to guess labels—but to reconstruct these hidden curves as accurately as possible.

GMM and Sequential Thinking

By itself, a Gaussian Mixture Model does not understand time, order, or sequence. It is not a temporal model. It simply analyzes data as a static distribution and tries to explain it using multiple Gaussian components.

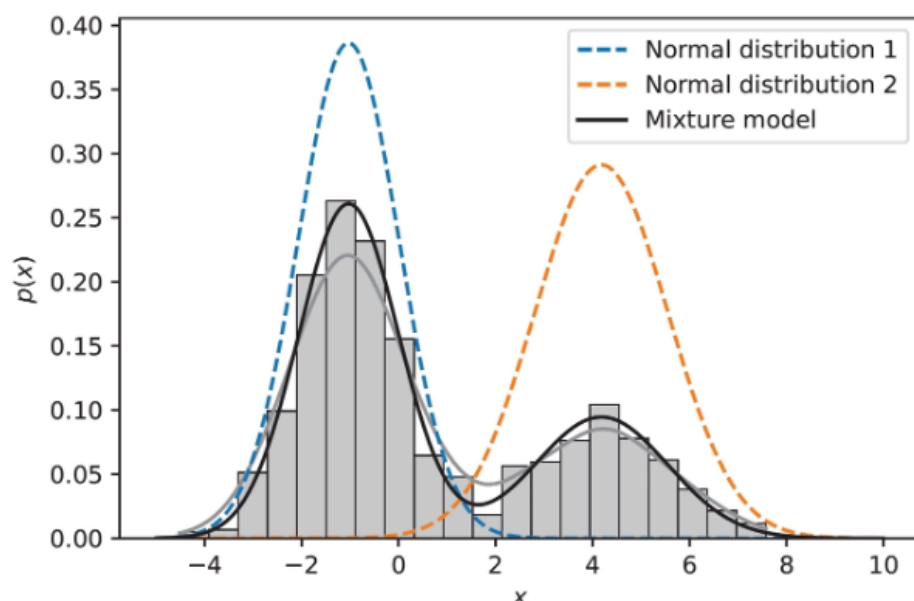
However, GMM becomes significantly more powerful when it is combined with sequential models like the **Hidden Markov Model (HMM)**. In such systems, the HMM handles *what state comes next*, while the GMM handles *what each state looks like statistically*.

This combination is widely used in areas like speech recognition, bioinformatics, and time-series analysis—anywhere the data has both structure and uncertainty layered on top of it.

Why multiple Gaussians matter

In real data, things rarely come from a single clean distribution. That idea is more of a mathematical comfort than a reflection of reality.

(Reference the image below)



Take student exam scores, for example. If all students had the same study habits, we might see a single smooth bell curve. But in reality, there are different underlying behaviors. Some students study consistently throughout the semester. Others only prepare shortly before the exam. These are not just noise differences—they are fundamentally different generative processes.

So instead of one bell curve, the data begins to look like multiple overlapping ones.

- One group centers around lower scores (less preparation)
- Another group centers around higher scores (consistent study)

A single Gaussian cannot represent both at the same time. It tries—and fails—because it is forced to pick one center, one variance, one identity for everything.

GMM solves this by allowing multiple hidden Gaussians to coexist. It does not try to flatten the structure into one explanation. It accepts that there are multiple sources underneath the data.

How GMM interprets a single point

Now the interesting part is not just discovering these distributions—but how the model *uses* them.

When a new value appears, say a score of 75, GMM does not immediately decide where it belongs. Instead, it evaluates the probability of that point under each Gaussian:

- It might compute: *“There is a 40% chance this came from Distribution 1.”* *“There is a 60% chance this came from Distribution 2.”*

This is called **soft assignment**, and it is one of the defining ideas behind GMM.

Unlike K-Means, which says *“you are here”*, GMM says *“you likely belong here... but also partially there”*. That small difference changes everything.

Because now clustering is no longer about drawing borders—it becomes about modeling uncertainty itself.

And in many real-world problems, uncertainty is not a flaw in the data. It is the truth of the data.

2. Gaussian Mixture Models - The Comprehensive Workflow

To understand Gaussian Mixture Models in practice, we will apply the idea to a familiar dataset: the **Iris dataset (or I would love to call it MY dataset)**—a classic benchmark used in clustering and classification tasks.

This dataset contains measurements of flowers described by four features: **sepal length; sepal width; petal length; petal width**. Each row is an observation of a flower, represented as a point in a 4-dimensional space:

sepal length	sepal width	petal length	petal width
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
7.0	3.2	4.7	1.4
6.4	3.2	4.5	1.5
6.9	3.1	4.9	1.5

sepal length	sepal width	petal length	petal width
5.5	2.3	4.0	1.3
6.3	3.3	6.0	2.5
5.8	2.7	5.1	1.9
7.1	3.0	5.9	2.1
6.3	2.9	5.6	1.8

Now suppose we want to cluster this dataset into **three groups** ($K = 3$).

The core idea of GMM in practice

A Gaussian Mixture Model does not assign a data point to a single cluster immediately. Instead, it treats cluster membership as something uncertain and probabilistic.

For every observation x_n , the model computes:

$$P(z_n = k \mid x_n)$$

This is called the **posterior probability** or **responsibility**, and it tells us how strongly cluster k explains the point x_n .

Using Bayes' idea, this probability is computed as:

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}$$

Where:

- z_n : hidden variable indicating the cluster assignment
- π_k : mixing weight of cluster k
- μ_k : mean vector of cluster k
- Σ_k : covariance matrix of cluster k
- $\mathcal{N}(x \mid \mu, \Sigma)$: Gaussian probability density function

The denominator is not just a mathematical detail, it ensures that all probabilities across clusters sum to 1, making the interpretation consistent.

A single observation: how the model “thinks”

Take the first data point: $x_1 = (5.1, 3.5, 1.4, 0.2)$

Now imagine the model has already formed three *hypothetical* Gaussian clusters:

- Cluster 1 → small petals
- Cluster 2 → medium petals
- Cluster 3 → large petals

The model does not immediately label the point. Instead, it evaluates:

- $P(z_1 = 1 \mid x_1)$ → Probability of belonging to Cluster 1
- $P(z_1 = 2 \mid x_1)$ → Probability of belonging to Cluster 2
- $P(z_1 = 3 \mid x_1)$ → Probability of belonging to Cluster 3

Each value represents *how strongly each hidden distribution explains this observation*.

Computing Gaussian likelihoods

To compute these probabilities, we first evaluate how likely the point is under each Gaussian distribution using the **multivariate Gaussian density function**:

$$\mathcal{N}(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

Where:

- d : number of features
- μ : mean vector
- Σ : covariance matrix
- $|\Sigma|$: determinant
- Σ^{-1} : inverse covariance matrix

This function is what allows the model to evaluate probabilities in a **multi-dimensional space**, where each flower is not just a point—but a structured object with relationships between features.

Thankfully, we do not need to plug in this formula by hand. We let Python help us with the **implementation of the Gaussian PDF**

```
import numpy as np

def gaussian_pdf(x, mu, sigma):
    d = len(x)

    det_sigma = np.linalg.det(sigma)
    inv_sigma = np.linalg.inv(sigma)

    diff = x - mu
    exponent = 1 / ((2 * np.pi) ** (d / 2) * (det_sigma **
0.5))
    fx = exponent * np.exp(-0.5 * diff.T @ inv_sigma @ dif
f)

    return fx
```

During the **Expectation step** of the EM algorithm, this function becomes the engine that computes how much each Gaussian “claims” responsibility for each data point.

Initializing the GMM (before learning begins)

At the start, the model does not know anything about the structure of the data. So we initialize the parameters in a simple, neutral way. These values are not final—they are just a starting assumption before refinement begins.

- **Initializing means:** The mean vector represents the center of each cluster. A practical initialization strategy is to randomly select real data points:

```
mu = X[np.random.choice(N, K, replace=False)]
```

This ensures that cluster centers start somewhere *inside* the data distribution instead of outside it, which helps the model stabilize faster.

- **Initializing covariance matrices:** The covariance matrix defines the *shape and orientation* of each Gaussian cluster. At the beginning, we assume no prior structure, so we initialize each cluster with an identity matrix:

```
sigma = [np.eye(d) for _ in range(K)]
```

This means:

- Features are initially treated as independent
- Variance is equal in all directions
- Clusters start as simple, circular shapes

Later, EM will reshape them into something meaningful.

- **Initializing mixing weights:** The mixing weight π_k represents how likely it is that any random point belongs to cluster k . We start with a neutral assumption: all clusters are equally likely.

For $K = 3$:

```
pi = np.ones(K) / K
```

So initially: $\pi_1 = \pi_2 = \pi_3 = \frac{1}{3}$

This is not a belief—it is a starting equilibrium before the model begins learning from data.

At this point, the GMM is not yet “intelligent.” It has no understanding of structure—it only has assumptions. But that is exactly the point. Because Gaussian Mixture Models do not start by knowing the answer. They start by assuming nothing, and slowly reconstruct structure through probability, iteration, and refinement. And that refinement process is where the real intelligence begins.

3. Gaussian Mixture Models - The Expectation–Maximization Algorithm

At this point, we have a model structure—but it is still untrained. The means, covariances, and mixing weights are only initial guesses. Nothing about them is

optimal yet.

So the real question becomes: *How does the model actually learn these parameters from data?*

The answer is the **Expectation–Maximization (EM) algorithm**, which is the core optimization procedure behind Gaussian Mixture Models.

Instead of solving everything in one step, EM takes an iterative approach. It repeatedly refines its understanding of the data by alternating between two complementary processes:

- **E-Step (Expectation):** Estimate how much each Gaussian component is responsible for each data point
- **M-Step (Maximization):** Update the parameters of the Gaussians based on those responsibilities

This cycle repeats until the model stabilizes—meaning the log-likelihood stops improving significantly.

In other words, the model gradually stops “guessing” and starts converging toward a stable explanation of the data.

E-Step (Expectation): Assigning responsibility

During the E-Step, the model computes something called the **responsibility**:

$$r_{nk} = P(z_n = k \mid x_n)$$

This value represents a soft assignment: *How much does cluster k explain data point x_n ?*

Unlike hard clustering methods, there is no single winner here. Every data point receives a probability distribution across all clusters.

So for each point:

- It might belong 70% to one cluster → 20% to another → 10% to another

And importantly, all responsibilities across clusters always sum to **1**.

This step is essentially the model asking: *“Given what I currently believe about the clusters, where does each point most likely belong?”*

M-Step (Maximization): Updating the model

Once responsibilities are computed, the model updates its parameters to better reflect the data structure it just inferred.

Now the question shifts: "If these responsibilities are correct, what should the clusters look like?"

- **Updating the mean** - The mean of each cluster is updated as a **weighted average** of all data points:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$$

Here, not all points contribute equally. Points with high responsibility for cluster k pull the mean toward themselves. Points with low responsibility have little influence

So the cluster center is not fixed—it is *pulled by the data it believes belongs to it*.

- **Updating the covariance matrix** - Next, the model updates the covariance matrix, which defines the **shape and orientation** of each Gaussian:

$$\Sigma_k = \frac{\sum_{n=1}^N r_{nk} (x_n - \mu_k)(x_n - \mu_k)^T}{\sum_{n=1}^N r_{nk}}$$

This step is where clusters start to physically reshape themselves. Instead of assuming spherical clusters, the model learns how stretched a cluster is, whether it is tilted, how features vary together. In essence, each Gaussian adjusts its geometry to better match the data it is responsible for.

- **Updating mixing weights** - Finally, the mixing weights are updated:

$$\pi_k = \frac{1}{N} \sum_{n=1}^N r_{nk}$$

This represents the **effective size of each cluster**. If a cluster explains many points strongly, its weight increases. If it explains fewer points, its weight decreases. So the model is continuously redistributing "importance" among clusters based on what it observes.

- **The objective: maximizing likelihood:** All of these updates are driven by one central idea: maximizing how well the model explains the data.

The likelihood of the full dataset under the Gaussian Mixture Model is:

$$L(\mu_k, \Sigma_k, \pi_k) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)$$

This expression represents the probability of observing the entire dataset given the current model parameters.

However, there is a practical issue here: multiplying many small probabilities together quickly leads to extremely tiny values, which can cause **numerical underflow**.

Switching to log-likelihood - To avoid this, we take the logarithm of the likelihood:

$$\log L = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k) \right)$$

This transformation does not change the optimization goal, but it makes the computation stable. Now instead of multiplying tiny numbers, we are summing log-probabilities, which is much more numerically reliable.

Convergence of the EM algorithm

One of the most important properties of EM is that it is guaranteed to **never decrease the log-likelihood** after each iteration. It either improves it or keeps it the same.

So the training process becomes a loop:

- Initialize μ_k, Σ_k, π_k
- E-Step: compute responsibilities
- M-Step: update parameters
- Compute log-likelihood
- Repeat

Over time, the model gradually stabilizes. The Gaussian components stop shifting significantly, and the structure of the data becomes fully captured by the mixture. At that point, the model is no longer guessing. It has converged to an explanation of the data that is statistically consistent under the Gaussian mixture assumption.

Expectation–Maximization Algorithm using Python

- We define a function that performs the full Expectation–Maximization procedure on a dataset:

```
def expectation_maximization(df, K=2, max_iter=100, tol=1e-4):
    # Parameters
    ...

    * df: input dataset (Pandas DataFrame)
    * K: number of clusters
    * max_iter: maximum number of iterations
    * tol: convergence threshold
    ...

    # Preparing the data
    X = df.values
    N, d = X.shape
    responsibilities = []
    # Initial parameters
    mu = X[np.random.choice(N, K, replace=False)]
    sigma = [np.eye(d) for _ in range(K)]
    pi = np.ones(K) / K
    log_likelihoods = []

    for iteration in range(max_iter):
        ...
```

Now, inside the iteration: We perform both the E-Step and the M-Step

- E-Step (Expectation): Computing responsibilities

```

# E-Step (Expectation): Computing responsibilities
responsibilities = np.zeros((N, K))

for n in range(N):
    numerator = []

    for k in range(K):
        value = pi[k] * gaussian_pdf(X[n], mu[k], sigma
[k])
        numerator.append(value)

    denominator = np.sum(numerator)

    for k in range(K):
        responsibilities[n, k] = numerator[k] / denominator
or

```

The above code compute the **responsibility matrix**, which stores: $r_{nk} = P(z_n = k \mid x_n)$. This represents how strongly each cluster explains each data point.

For each data point:

- The model evaluates how likely it is under each Gaussian
- It multiplies this likelihood by the cluster's weight π_k
- Then normalizes across all clusters

This produces a probability distribution over clusters for every point. So instead of saying: "This point belongs to cluster 1" → The model says: "This point is 70% cluster 1, 20% cluster 2, 10% cluster 3". That is the essence of **soft clustering**.

- **M-Step (Maximization): Updating parameters**

```

# M-Step (Maximization): Updating parameters
Nk = np.sum(responsibilities, axis=0)

```

```

mu_new = []

for k in range(K):
    weighted_sum = np.sum(responsibilities[:, k].reshape(
        -1, 1) * X, axis=0)
    mu_k = weighted_sum / Nk[k]
    mu_new.append(mu_k)

mu = np.array(mu_new)

```

Once responsibilities are computed, the model updates the Gaussian parameters.

- The first line defines the **cluster size** → **This gives the effective number of points assigned to each cluster.**
 - In the loop, the cluster center is updated as a weighted average
 - Points with higher responsibility influence the mean more
 - The cluster “moves” toward the region it explains best
 - So the mean is no longer fixed—it is *pulled by the data it believes belongs to it*.
- **M-Step (Maximization): Updating covariance matrices**

```

sigma_new = []

for k in range(K):
    sigma_k = np.zeros((d, d))

    for n in range(N):
        diff = (X[n] - mu[k]).reshape(-1, 1)
        sigma_k += responsibilities[n, k] * (diff @ diff.T)

    sigma_k = sigma_k / Nk[k]

    sigma_k += 1e-6 * np.eye(d) # regularization
    sigma_new.append(sigma_k)

```

```
sigma = sigma_new
```

The covariance defines the **shape and spread** of each cluster.

The model learns how features vary together. Clusters can stretch, rotate, and deform. Regularization avoids numerical instability. So each Gaussian gradually becomes a flexible shape that fits the data structure.

- **M-Step (Maximization): Updating mixing weights**

```
# The mixing weights
pi = Nk / N
```

This updates the probability that a random point belongs to each cluster. So:
Large clusters get higher weight → Small clusters get lower weight

- **Log-likelihood: measuring model quality**

To track progress, we compute the log-likelihood:

```
# Log Likelihood
log_likelihood = 0

for n in range(N):
    total = 0

    for k in range(K):
        total += pi[k] * gaussian_pdf(X[n], mu[k], sigma[k])

    log_likelihood += np.log(total)

log_likelihoods.append(log_likelihood)
```

The `gaussian_pdf()` function earlier → I hope you remember it

Why log-likelihood? The original likelihood multiplies many small probabilities:

$$L = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_n)$$

This quickly becomes numerically unstable. So we use: $\log L$. This transforms multiplication into addition, making computation stable and easier to optimize.

- **Convergence check**

```
if iteration > 0:
    if abs(log_likelihoods[-1] - log_likelihoods[-2]) <
tol:
        print(f"Converged at iteration {iteration}")
        break
```

The algorithm stops when improvement becomes negligible. So the process becomes: update responsibilities → update parameters → check improvement → repeat.

Until nothing significant changes anymore.

- **Outside the loop, we return the results**

```
return mu, sigma, pi, responsibilities, log_likelihoods
```

The model returns: final cluster centers, covariance matrices, mixing weights, responsibility matrix, training history

```

import pandas as pd

data = {
    "sepal.length": [5.1, 4.9, 4.7, 4.6, 7.0, 6.4, 6.9, 5.
5, 6.3, 5.8, 7.1, 6.3],
    "sepal.width": [3.5, 3.0, 3.2, 3.1, 3.2, 3.2, 3.1, 2.
3, 3.3, 2.7, 3.0, 2.9],
    "petal.length": [1.4, 1.4, 1.3, 1.5, 4.7, 4.5, 4.9, 4.
0, 6.0, 5.1, 5.9, 5.6],
    "petal.width": [0.2, 0.2, 0.2, 0.2, 1.4, 1.5, 1.5, 1.
3, 2.5, 1.9, 2.1, 1.8],
}

df = pd.DataFrame(data)

mu, sigma, pi, responsibilities, log_likelihoods = expectation_maximization(df, K=3)

print("Mean:\n", mu)
print("Sigma:\n", sigma)
print("Pi:\n", pi)
print("Responsibilities:\n", responsibilities)
print("Log Likelihoods:\n", log_likelihoods)

```

I will not show the output here due to the length of it

After running EM, the model converges in a few iterations (for example, around 7 in this case). But the most important object is not the final means or covariances. It is the **responsibility matrix**. Each row corresponds to a data point, and each column corresponds to a cluster:

Sample	Cluster 1	Cluster 2	Cluster 3
1	1.000	0.000	~0
2	1.000	0.000	0
5	0.000	0.000	1.000
8	0.000	1.000	0

Even though the model is probabilistic, the values often become close to 0 or 1 when clusters are well separated. That means the model has discovered very clear structure in the data.

Why responsibilities keep changing at each iteration: Means shift toward weighted centers → Covariances reshape the clusters → Mixing weights adjust proportions

And because the parameters change, the responsibilities change too. So EM becomes a feedback loop: **responsibilities** → **update parameters** → **new parameters** → **new responsibilities**. This cycle continues until stability is reached.

- **Final behavior of the model**

During training, the log-likelihood steadily increases:

$$33.31 \rightarrow -14.66 \rightarrow -9.65 \rightarrow \dots \rightarrow 31.37$$

This confirms that each iteration improves the model's explanation of the data.

- **Final cluster assignment**

Once training is complete, we assign each point to the most likely cluster:

```
clusters = np.argmax(responsibilities, axis=1)
df["Cluster"] = clusters

print(df)
```

Here: Cluster 0 → first group; Cluster 1 → second group; Cluster 2 → third group

[You can download the full Gaussian Mixture by hand code here:](#)

[gaussian_mixture.py](#)

The EM algorithm is not about making a single decision in one step. It is a process of gradual refinement. Each iteration slightly adjusts: what the model believes clusters look like; how data points relate to those clusters, and how confident those relationships are.

Over time, uncertainty turns into structure. Not because the model was told the answer—but because it slowly inferred it from probability alone.

4. Gaussian Mixture Models with Python

Gaussian Mixture Models with Python is straightforward with the Scikit-Learn library:

```
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
```

```
from sklearn.mixture import GaussianMixture

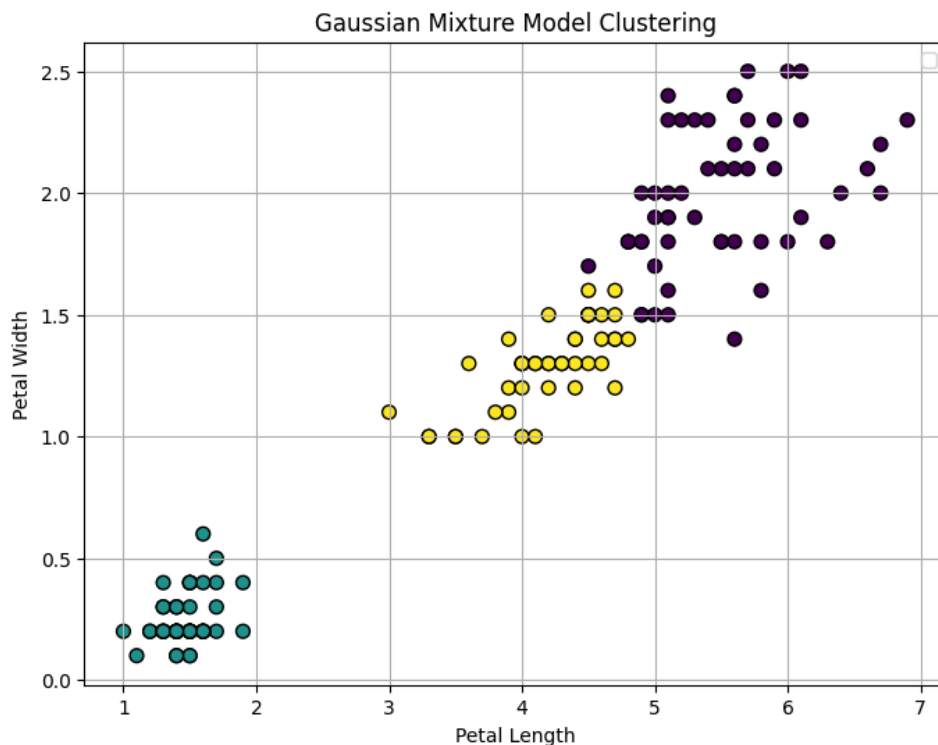
# Train the Gaussian Mixture model
model = GaussianMixture(n_components=3, random_state=42)
model.fit(X)

# Predict the cluster labels
labels = model.predict(X)
```

```
import matplotlib.pyplot as plt

# Plot the clusters
plt.figure(figsize=(8, 6))
plt.title("Gaussian Mixture Model Clustering")
plt.scatter(X[:,2], X[:,3], c=labels, cmap='viridis', s=50,
            edgecolor='k')
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.grid(True)
```

```
plt.legend()  
plt.show()
```



Summary

Gaussian Mixture Models are still clustering models—but they approach the problem from a completely different perspective. Instead of relying purely on **distance** like K-Means or hierarchical clustering, or focusing on **density** like DBSCAN, GMM looks at the data through the lens of **probability distributions**.

Rather than drawing rigid boundaries or grouping points based on proximity alone, GMM assumes that the data is generated from multiple underlying Gaussian distributions—and its goal is to uncover those hidden structures.

This leads to a more flexible interpretation of clustering:

- Points are not forced into a single cluster → they belong with certain probabilities
- Clusters are not fixed shapes → they adapt to the data through covariance
- Structure is not imposed → it is *inferred* from how likely the data fits different distributions

So while other methods ask: "How close is this point to a cluster?" → GMM asks: "Which distribution most likely generated this point—and by how much?"

That shift—from distance to probability—is what makes Gaussian Mixture Models both more expressive and, in many cases, closer to how real-world data actually behaves.

References

<https://www.geeksforgeeks.org/machine-learning/gaussian-mixture-model/>

<https://www.ibm.com/think/topics/gaussian-mixture-model>

<https://www.youtube.com/watch?v=wT2yLNUfyoM>

<https://www.youtube.com/watch?v=C7jhwN6H9LU>

Next Section:

 [42. Apriori Algorithm](#)